Understanding exactly-once processing and windowing in streaming pipelines

Israel Herraiz ihr@google.com @herraiz

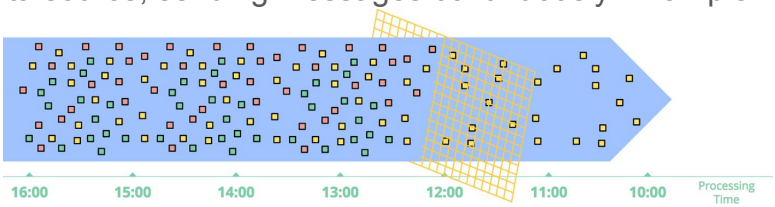
Apache Beam Summit, August 25th 2020

This version of the slides contain some additional slides compared to what is included in the video.

These additional slides contain some more details about how the unit test was created, and are included for completeness of the slides as a reusable document.

Stream processing

Unbounded data source, sending messages continuously. Example:



Potential problems: duplicates, dropping messages (e.g. late data)

This talk:

- How to evaluate whether we may be dropping data?
- Understanding windowing for exactly-once processing with order even with at-least once delivery and no guarantees of delivery order

Figure: http://streamingsystems.net/fig/1-5

In streaming pipelines, data inputs will be unbounded, sending messages continuously.

We may think that the main problem is keeping up with a high load, so we have enough capacity to process all messages.

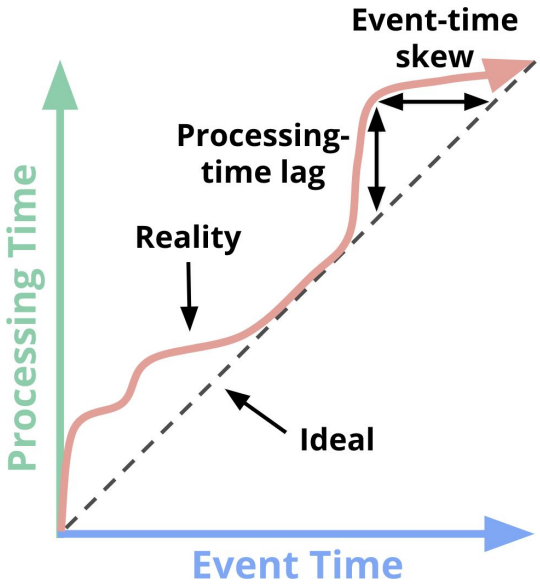
But there are other problems too: the source may send the same message more than once, and we may drop messages, depending on how we process them in Apache Beam.

In this talk we will see in which situations we may be dropping data and how to evaluate whether we might be dropping data to make sure we don't drop any.

We will see that with Apache Beam, we can make exactly once processing, using the original order of messages, even in situations where the source is ensuring only at-least once delivery, and cannot guarantee the order of delivery.

What is late data?

Two dimensions of time



http://streamingsystems.net/fig/1-1

- Two time dimensions
  - **Event time**
  - **Processing time**
- **Watermark**: relationship between both
  - Estimated as timestamp of oldests message seen in the input message queue (waiting to be processed)
  - When we process messages, the watermark advances

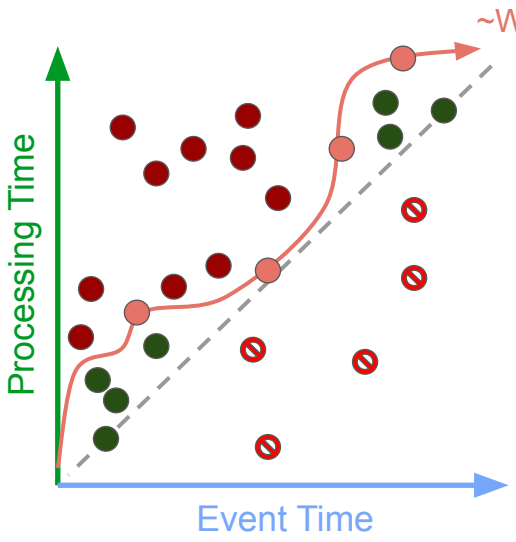
In the previous slide we have shown a stream of data. We think of streams of data as uni-dimensional sources.

The data is sent over time, and there are just measurements over time. But in reality, we have 2 dimensions of time:

Event time and Processing time

Event time is the original timestamp when the message originated.
Processing time is when we actually see that message in our pipeline. For different reasons, processing time will always be higher than event time.
The relationship between event time and processing time defines a curve, called "the watermark".
Because we are working in streaming, we don't know if whether we will see some late messages coming, so the actual watermark is an estimation (as the timestamp of the oldest message seen in the input message queue)

What is late data?



- **<u>Arriving after the watermark? Then you are late</u>**
- Late date is defined comparing the event time against the watermark
- A message is not late *per-se*, it is late compared against the watermark (i.e. the rest of previously seen messages)

Late messages
On-time messages
Early messages
Impossible message (processing time < event time)

So data is not late per-se. Data is late when compared against other messages.

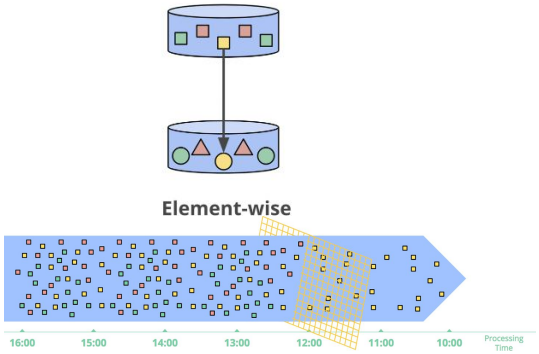
The event timestamp of a message does not define alone whether the message is late. When we compare with the watermark, it is when we say whether the message is early or late.

In Apache Beam, depending on the kind of calculations that we do with the data, we will have to make decisions about how to deal with late data.

Some of those decisions may drop data.

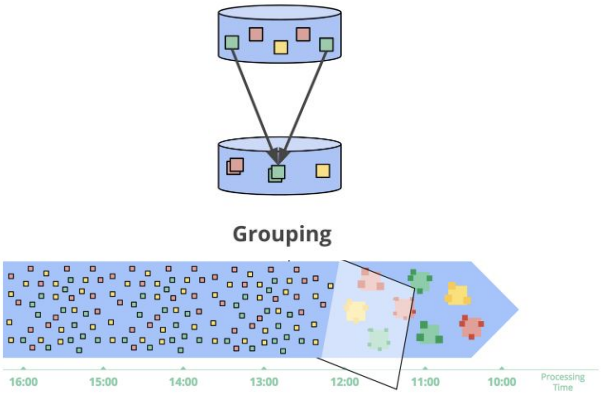
In Apache Beam

No aggregations? No problem. Don't even bother about event time, processing time, late data or duplicates reprocessing. All Beam runners will make a decent job.



http://streamingsystems.net/fig/1-5 http://streamingsystems.net/fig/1-7
http://streamingsystems.net/fig/2-2

Aggregating? You may drop late data, because you will need to apply a window. Need to decide for how long you will wait for late data.



If in our pipeline, we don't make aggregations, we then basically don't care about event or processing time, or late data.

We may have issues with duplicates, but most likely, any Apache Beam runner will make a decent job making sure that duplicates are filtered. If the message queue re-delivers a message, the runner should be able to detect and make sure the message is not processed again.

However, if we aggregate, we will need to apply a window to the stream. When applying a window, we need to make a decision about how to deal with late data, and that decision will imply dropping late data under certain conditions.

And this applies regardless the type of window you want to apply. It is the same for any kind of windows.

Let's go step by step.

First step: reading data from a stream

But let's go step by step.

First we need to read data. You will see that despite the importance of event time, most likely, you have been using processing time by default with your streaming pipelines.

Reading data from streaming sources

In streaming, messages are **timestamped**

- https://beam.apache.org/documentation/transforms/python/elementwise/withtimestamps/
- https://beam.apache.org/releases/javadoc/2.23.0/org/apache/beam/sdk/values/TimestampedValue.html

If we don't specify anything, your IO module will use *processing time* timestamps

- Google Cloud Pub/Sub (PubsubIO)
- Apache Kafka (KafkaIO)

How can we use *event time* processing with our stream?

When reading from a streaming data source, we are producing *timestamped values*.

That is, in addition to the type we are reading (strings, etc), each message will also have a timestamp associated to it

If we don't specify anything, it is likely that we are using processing time. That's the case of PubsubIO and KafkaIO in Apache Beam.

So, then, how can we use event time instead?

First approach: parse the timestamp and add it

`MyDummyEvent.getEventTimestamp` returns a `Long`

```
PCollection<String> messages =
    p.apply( name: "Read from Pubsub", PubsubIO.readStrings().fromSubscription(subscription));

PCollection<MyDummyEvent> events =
    messages.apply(
         name: "Parse",
        MapElements.into(TypeDescriptor.of(MyDummyEvent.class)).via(MyDummyEvent::fromString));

PCollection<MyDummyEvent> withTimestamps =
    events.apply(
         name: "Add event timestamps",
        WithTimestamps.of(e -> Instant.ofEpochMilli(e.getEventTimestamp())));
```

Read

Parse

Add timestamp

Let's focus on the case of Google Cloud Pubsub. For Kafka, it would work in a very similar way.

In Pubsub, messages are just strings of bytes. Typically, the timestamp, produced in the original source as part of the message, will be part of that string of bytes.

So how can we decode that to be used with our stream?

If we know that the timestamp is inside the messages, we can just read them, parse the timestamps, and apply them.

This works with any kind of streaming source.

After adding the timestamps, we could apply a window, reasoning in event time.

Second approach: use the topic attributes

If you are using the Google Cloud SDK

- Subscribe to
  projects/**pubsub-public-data**/topics/**taxirides**-realtime
- Grab some messages and look at the attributes of the messages

	MESSAGE_ID	ATTRIBUTES	DELIVERY_ATTEMPT
ht":1}	1206646531578961	ts=2020-05-20T14:55:49.51875-04:00	


```
{
  "ride_id": "f172ff2a-b6fc-4345-b6df-d8e3c207b89c",
  "point_idx": 817,
  "latitude": 40.81768,
  "longitude": -73.94772,
  "timestamp": "2020-05-20T14:55:48.85921-04:00",
  "meter_reading": 17.08353,
  "meter_increment": 0.020910075,
  "ride_status": "enroute",
  "passenger_count": 1
}
{
  "ride_id": "c87a5a0e-4c75-4e12-8b8c-d3823cd31da2",
  "point_idx": 263,
  "latitude": 40.76919,
  "longitude": -73.91166000000001,
  "timestamp": "2020-05-20T14:55:48.86264-04:00",
  "meter_reading": 5.7470374,
  "meter_increment": 0.021851853,
  "ride_status": "enroute",
  "passenger_count": 1
}
{
  "ride_id": "2f9c21cb-b116-4219-a775-51edcc3c7531",
  "point_idx": 522,
  "latitude": 40.69608,
  "longitude": -73.98343000000001,
  "timestamp": "2020-05-20T14:55:48.86356-04:00",
  "meter_reading": 10.735218,
  "meter_increment": 0.020565553,
  "ride_status": "enroute",
  "passenger_count": 2
}
```


In Pubsub, messages can be published along with attributes.

Attributes are a map of key-values. At the moment of publishing the message, we can use one of the keys to write a timestamp, and then use that timestamp in our pipeline.

In the right side, we show some parsed messages, and we can check that timestamp attribute in the topic attributes is the same as the one that was included inside the message when it was published (some minor differences in the picture because it is not exactly the same messages)

Second approach: use the topic attributes

https://beam.apache.org/releases/javadoc/2.23.0/org/apache/beam/sdk/io/gcp/pubsub/PubsubIO.Read.html#withTimestampAttribute-java.lang.String-

```
PCollection<String> messages =
    p.apply(
         name: "Read from Pubsub",
        PubsubIO.readStrings().fromSubscription(subscription).withTimestampAttribute("ts"));

PCollection<MyDummyEvent> events =
    messages.apply(
         name: "Parse",
        MapElements.into(TypeDescriptor.of(MyDummyEvent.class)).via(MyDummyEvent::fromString));
```

Read with timestamp

Parse

*And if you are using KafkaIO, you have similar methods to set the timestamp of messages.
For instance: KafkaIO.Read.withCreateTime

If the timestamp attributed is properly formatted, we can just use it to create timestamped values, and work with event time stream processing after that.

This approach differs for each message queue. For instance, for Kafka we could use withCreateTime, or other options, depending on how we want to set the timestamp of the messages.

In the code snippets shown previously, we have zero risk of dropping any data.

It is only when we apply a window when we assume a risk of dropping late data.

What's the difference between event and processing time?

	Event time	Processing time
Complex Event Processing (e.g. recover order of data inside the window)	Yes (even with message queues with no order guarantees)	No
Possibility of dropping data when we apply a window	Yes (but can be managed)	No (simple calculations, simple message handling)

In our pipeline, depending on what we want to do, we should decide whether we want to work on event time or processing time.

If we don't do aggregations, or we don't make a calculation that requires knowing the original event timestamp, we could try to just use processing time. By definition, there is no possibility of late data if we work on processing time, so we would never drop any message even if we apply a window. But we would be limited in the kind of calculations we could do.

If we aggregate, or if we want to make calculations that require the original event timestamp, we should use event time, and manage how to work with late data. Using event time will allow us for instance recovering the original order of the data, even if the message queue does not guarantee the order of delivery, or if the message queue is at least once only.

If we work on event time,
we will be able to apply
complex logic/business rules
to our streams,
offering results with low latency

Let's assume we work on event time.

Second step: applying a window

So working on processing time is simple, but really not so interesting.

We want to make complex calculations and business rules, to a stream of data, with a high volume of messages, low latency, etc.

Using windowing we can do complex aggregations and calculations, but we risk dropping data.

Let's see how to assess and handle that risk.

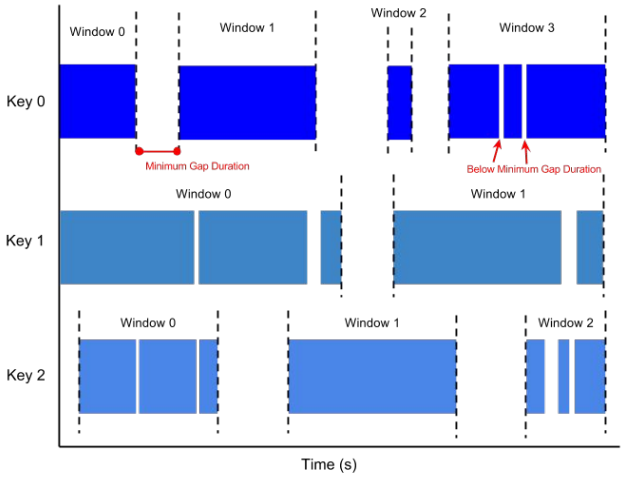
Windows in Apache Beam

The basics are well covered in sources such as:

- The Beam Programming Guide
- The book *Streaming Systems* http://streamingsystems.net/

Using windows, triggers, late data, etc, can be confusing.

- How can we make sure that window is working in the way we intend?
- How can we make sure that we will not drop data applying a window?



There are different kinds of windows in Apache Beam, and several other concepts such as triggers, timestamps, etc, that are well covered in the usual documentation.

However, it is true that using windows, triggers, late data, accumulation modes, etc, can be sometimes confusing.
In a pipeline, we may think that we are not dropping any data, or that we are producing output with a certain latency or cadence, and then discover that this is not the case, when the pipeline is running with a streaming source.
How can we ensure that the behavior of our streaming pipeline is the one that we intend? How can we check if our streaming pipeline

Unit testing to the rescue

https://github.com/iht/beam-late-data

With *TestStream* you can control the order of delivery of messages, the values of the watermark, and other properties:

> TestStream (Apache Beam Java SDK Docs)
>
> https://beam.apache.org/blog/test-stream/

We are generating 50 on-time and 10 late messages

- We *make* the messages late by controlling the watermark
  - **Remember:** messages are not late per-se, the are late only when compared against the watermark

Let me present this personal Github repo, with a unit test that showcases how to test whether a window will drop late data.

The repo uses the TestStream class to control how the messages are delivered, and the values of the watermark.

TestStream allows you to test, in a deterministic way, a streaming pipeline.

In the case of this repo, I am generating 60 messages in total: 50 on-time messages and 10 late messages. To make messages late, I control the value of the watermark, making sure that the event time of my messages is lower than the watermark.

Approach to producing late data

```
// Add on-time events
Instant currentWatermark = TEST_EPOCH;
for (int k = 0; k < N_ONTIME; k++) {
  currentWatermark = estimateWatermark(onTimeEvents, k);
  base =
      base.addElements(onTimeEvents.get(k))
          .advanceProcessingTime(Duration.standardSeconds(2))
          .advanceWatermarkTo(currentWatermark);
}

currentWatermark = currentWatermark.plus(Duration.standardSeconds(2));

// Wait some processing time
base = base.advanceProcessingTime(Duration.standardSeconds(3));
base.advanceWatermarkTo(currentWatermark);
```

We add messages and advance the watermark accordingly.

We estimate the watermark as the oldest timestamp of the messages still waiting to be processed. The messages collection is in no particular order.

```
// Add late data
// This data is late because their timestamps are older than the watermark
// We increase the watermark with every late message by 2 seconds.
for (int k = 0; k < N_LATE; k++) {
  currentWatermark = currentWatermark.plus(Duration.standardSeconds(2));
  base =
      base.addElements(lateEvents.get(k))
          .advanceProcessingTime((Duration.standardSeconds(2)))
          .advanceWatermarkTo(currentWatermark);
}

// Let's finalize the stream
TestStream<KV<String, MyDummyEvent>> st =
    base.advanceProcessingTime((Duration.standardSeconds(2))).advanceWatermarkToInfinity();
```

We send another stream of messages once the watermark has passed.

These messages will be late data. We advance the watermark by 2 seconds every time, rather than estimate it

To produce late data, we create two collections of messages, with similar event times.

We iterate over the first collection, and we make the watermark advance "by hand".

Once we have finished traversing the first collection, and the watermark has progressed, we traverse the second collection and the messages to the TestStream.

This second wave of messages will be late data.

The TestStream can now be used with any pipeline to reason about late data.

Testing whether we are dropping late data or not

Conservation of messages

- We can count messages in the stream, before applying the window
  - This will include any late message too (remember: messages are only potentially dropped when we apply a window)
- We count messages after applying a window, while we perform some aggregation
  - If there is late dropped, it would not be used in the aggregation, and therefore not counted
  - **Beware**: your pipeline can trigger the calculation several times, **keep a set** of seen messages, **not just a counter**, or you will over represent data

Testing a streaming pipeline

```
/** Streaming pipeline */
PCollection<KV<String, MyDummyEvent>> events = pipeline.apply(st);
// Identity, just to count elements
PCollection<KV<String, MyDummyEvent>> identity =
    events.apply(
        "Identity transform (to count processed events)",
        MapElements.into(
                TypeDescriptors.kvs(
                    TypeDescriptor.of(String.class), TypeDescriptor.of(MyDummyEvent.class)))
            .via(
                (KV<String, MyDummyEvent> kv) -> {
                  AggAndCountWindows.NUM_PROCESSED_EVENTS_BEFORE_WINDOW++;
                  return kv;
                }));
```

```
PCollection<KV<String, MyDummyEvent>> windowed = identity.apply(new SampleSessionWindow());

// We group, and then we will sum. We should get as many partial sums as triggers
PCollection<KV<String, Iterable<MyDummyEvent>>> grouped =
    windowed.apply("Group", GroupByKey.create());

// Keep track of window id for each trigger, count number of different windows and number of
// triggers
PCollection<PaneGroupMetadata> summed =
    grouped.apply("Sum groups", ParDo.of(new AggAndCountWindows()));
```

Read and count all messages
(including late messages)

Apply window, group and calculate aggregation.

While aggregating, keep track of the messages being used in the aggregation.

On the left side, we are using the TestStream and using an identify function just to increase a count of the messages seen in that stream.

Because we are not applying any window nor doing any aggregation, this count will include all messages, including any potential late message.

Then we apply some windowing, with some parameters to handle late data, group and apply and aggregation.

In that aggregation, we keep a set of all seen messages. Later, we will use that set to make sure that its cardinality matches the count done in the previous code snippet.

Use assertions to make sure that no late data is dropped

```
/** Test assertions */
List<TimestampedValue<KV<String, MyDummyEvent>>> allEvents = new ArrayList<>();
allEvents.addAll(onTimeEvents);
allEvents.addAll(lateEvents);

// Make sure all messages are processed before windowing
assertEquals(
    "All messages are processed before windowing",
    N_ONTIME + N_LATE,
    AggAndCountWindows.NUM_PROCESSED_EVENTS_BEFORE_WINDOW);

// Make sure all messages are counted after windowing (no late messages are dropped)
assertEquals(
    "No late messages are dropped after windowing",
    AggAndCountWindows.NUM_PROCESSED_EVENTS_BEFORE_WINDOW,
    AggAndCountWindows.SEEN_EVENTS_IN_TRIGGERS.size());
```

Compare the counts before and after

Finally, after the pipeline, we make sure that the counts match.

If we are using a window that would drop late data, the test would fail.

If we generate data with properties similar to the stream we want to use, we can check whether the window we would be using would drop data.

We can also explore the limits and check what is our tolerance for late data.

See it in action

Will [this window] drop late data?

Run `mvn test`

```
Window.<KV<String, MyDummyEvent>>into(FixedWindows.of(Duration.standardSeconds(100)))
    .triggering(
        AfterWatermark.pastEndOfWindow()
            .withEarlyFirings(
                AfterProcessingTime.pastFirstElementInPane()
                    .plusDelayOf(Duration.standardSeconds(5)))
            .withLateFirings(AfterProcessingTime.pastFirstElementInPane()))
    .withAllowedLateness(Duration.standardDays(10))
    .accumulatingFiredPanes();
```

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.google.cloud.pso.LateDropOrNotTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.054 s - in
 com.google.cloud.pso.LateDropOrNotTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

A CSV output with all the trigger results is written to `target/output/<RUN TIMESTAMP>/`

This is a naive example (10 days of allowed lateness).

The test passes, so not late data is dropped.

Inspecting triggers: fixed windows, no drop of late data

```
./scripts/show_data.sh target/output/2020-08-22T21:30:12.946Z
```

triggers	window	is_first	is_last	timing	n_before	n_after	N
1	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	True	False	EARLY	3	3	3
2	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	6	6	6
3	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	9	9	9
4	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	12	12	12
5	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	15	15	15
[...]							
10	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	30	30	30
11	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	33	33	33
12	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	36	36	36
13	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	39	39	39
14	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	42	42	42
15	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	45	45	45
16	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	48	48	48
17	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	ON_TIME	50	49	49
18	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	51	50	50
19	[2017-02-03T10:38:20.000Z..2017-02-03T10:40:00.000Z)	True	False	EARLY	51	51	1
20	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	52	52	51
21	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	53	53	52
22	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	54	54	53
23	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	55	55	54
24	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	56	56	55
25	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	57	57	56
26	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	58	58	57
27	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	59	59	58
28	[2017-02-03T10:36:40.000Z..2017-02-03T10:38:20.000Z)	False	False	LATE	60	60	59
29	[2017-02-03T10:38:20.000Z..2017-02-03T10:40:00.000Z)	False	True	ON_TIME	60	60	1

With this script we can check the output of the trigger, and study how our window is behaving.

In this case, we have used FixedWindows, and we can see that there are two different windows (blue and yellow). In the first window, we have some late data, in the second window, there was not late data.

The ON_TIME trigger is the one done at the watermark.

Because the test was successful, we are sure that no late data has been dropped (the total count is 60)

See it in action

Will this window drop late data?

```
Window<KV<String, MyDummyEvent>> window = Window.<KV<String, MyDummyEvent>>into(FixedWindows.
        .triggering(
                AfterWatermark.pastEndOfWindow()
                        .withEarlyFirings(
                                AfterProcessingTime.pastFirstElementInPane()
                                        .plusDelayOf(Duration.standardSeconds(5)))
                        .withLateFirings(AfterProcessingTime.pastFirstElementInPane()))
        .withAllowedLateness(Duration.standardSeconds(0))
        .accumulatingFiredPanes();
```

Run `mvn test`

```
java.lang.AssertionError: No late messages are dropped after windowing
Expected :60
Actual   :50
```

Another example, that drops late data (0 seconds of allowed lateness, which is the value by default if we don't set it), so the count does not match.

Another window: late data dropped

Colors alternating for different windows

triggers	window	is_first	is_last	timing	n_before	n_after	N
1	[2017-02-03T10:37:30.000Z..2017-02-03T10:37:40.000Z)	True	False	EARLY	3	3	3
2	[2017-02-03T10:37:30.000Z..2017-02-03T10:37:40.000Z)	False	False	EARLY	6	6	6
3	[2017-02-03T10:37:30.000Z..2017-02-03T10:37:40.000Z)	False	False	EARLY	9	8	8
4	[2017-02-03T10:37:30.000Z..2017-02-03T10:37:40.000Z)	False	True	ON_TIME	9	8	8
5	[2017-02-03T10:37:40.000Z..2017-02-03T10:37:50.000Z)	True	False	EARLY	10	10	2
6	[2017-02-03T10:37:40.000Z..2017-02-03T10:37:50.000Z)	False	False	EARLY	13	13	5
7	[2017-02-03T10:37:40.000Z..2017-02-03T10:37:50.000Z)	False	False	EARLY	16	16	8
8	[2017-02-03T10:37:40.000Z..2017-02-03T10:37:50.000Z)	False	False	EARLY	19	18	10
9	[2017-02-03T10:37:40.000Z..2017-02-03T10:37:50.000Z)	False	True	ON_TIME	19	18	10
10	[2017-02-03T10:37:50.000Z..2017-02-03T10:38:00.000Z)	True	False	EARLY	21	21	3
11	[2017-02-03T10:37:50.000Z..2017-02-03T10:38:00.000Z)	False	False	EARLY	24	24	6
12	[2017-02-03T10:37:50.000Z..2017-02-03T10:38:00.000Z)	False	False	EARLY	27	27	9
13	[2017-02-03T10:37:50.000Z..2017-02-03T10:38:00.000Z)	False	True	ON_TIME	29	28	10
14	[2017-02-03T10:38:00.000Z..2017-02-03T10:38:10.000Z)	True	False	EARLY	31	31	3
15	[2017-02-03T10:38:00.000Z..2017-02-03T10:38:10.000Z)	False	False	EARLY	34	34	6
16	[2017-02-03T10:38:00.000Z..2017-02-03T10:38:10.000Z)	False	False	EARLY	37	37	9
17	[2017-02-03T10:38:00.000Z..2017-02-03T10:38:10.000Z)	False	True	ON_TIME	38	37	9
18	[2017-02-03T10:38:10.000Z..2017-02-03T10:38:20.000Z)	True	False	EARLY	40	40	3
19	[2017-02-03T10:38:10.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	43	43	6
20	[2017-02-03T10:38:10.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	46	46	9
21	[2017-02-03T10:38:10.000Z..2017-02-03T10:38:20.000Z)	False	False	EARLY	49	48	11
22	[2017-02-03T10:38:10.000Z..2017-02-03T10:38:20.000Z)	False	True	ON_TIME	49	48	11
23	[2017-02-03T10:38:20.000Z..2017-02-03T10:38:30.000Z)	True	False	EARLY	50	50	2
24	[2017-02-03T10:38:20.000Z..2017-02-03T10:38:30.000Z)	False	True	ON_TIME	54	**50**	2

Only 50 messages

The alternating colors are different windows.

There is no sign of late data. Also, there are only 50 messages, and the stream had initially 60, so late data has been dropped.

The last 6 messages were all late, so only 54/60 messages have been included in the output. There was no trigger for the last 6 messages, they were just dropped.

A session window example

```
        Window.<KV<String, MyDummyEvent>>into(
                Sessions.withGapDuration(Duration.standardSeconds(5)))
            .triggering(
                AfterWatermark.pastEndOfWindow()
                    .withEarlyFirings(
                        AfterProcessingTime.pastFirstElementInPane()
                            .plusDelayOf(Duration.standardSeconds(8)))
                    .withLateFirings(AfterProcessingTime.pastFirstElementInPane()))
            .withAllowedLateness(Duration.standardDays(10))
            .accumulatingFiredPanes();
```

triggers	window	is_first	is_last	timing	n_before	n_after	N
1	[2017-02-03T10:37:32.781Z..2017-02-03T10:37:42.088Z)	True	False	EARLY	5	5	5
2	[2017-02-03T10:37:32.781Z..2017-02-03T10:37:47.164Z)	True	False	EARLY	10	10	10
3	[2017-02-03T10:37:32.781Z..2017-02-03T10:37:52.274Z)	True	False	EARLY	15	15	15
4	[2017-02-03T10:37:32.781Z..2017-02-03T10:37:56.651Z)	True	False	EARLY	20	20	20
5	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:02.456Z)	True	False	EARLY	25	25	25
6	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:06.435Z)	True	False	EARLY	30	30	30
7	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:12.365Z)	True	False	EARLY	35	35	35
8	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:16.925Z)	True	False	EARLY	40	40	40
9	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:21.698Z)	True	False	EARLY	45	45	45
10	[2017-02-03T10:37:32.781Z..2017-02-03T10:38:25.315Z)	True	False	EARLY	50	50	50
11	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	True	False	ON_TIME	52	52	52
12	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	53	53	53
13	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	54	54	54
14	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	55	55	55
15	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	56	56	56
16	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	57	57	57
17	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	58	58	58
18	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	59	59	59
19	[2017-02-03T10:37:32.298Z..2017-02-03T10:38:25.315Z)	False	False	LATE	60	60	60

Here we see how to match the output with the window we have applied. In this case, there is only one window, that expands everytime it finds a new message in the same session.

Notice how the window is expanded only until the watermark. Before the watermark, each new message changes the window id, its boundaries are expanded to the right.

When we reach the watermark, the window boundaries don't change anymore.

Also, we see that the late firing with the AfterWatermark trigger is fired for every late message.

How does AfterEach.inOrder work?

```
Window.<KV<String, MyDummyEvent>>into(Sessions.withGapDuration(Duration.standardSeconds(5)))
            .triggering(
                AfterEach.inOrder(
                        AfterPane.elementCountAtLeast(3),
                        AfterPane.elementCountAtLeast(10),
                        Repeatedly.forever(AfterPane.elementCountAtLeast(15))
                ))
            .withAllowedLateness(Duration.standardDays(10))
            .accumulatingFiredPanes();
```

triggers	window	is_first	is_last	timing	n_before	n_after	N
1	[2017-02-03T10:37:31.691Z..2017-02-03T10:37:40.797Z)	True	False	EARLY	3	3	3
2	[2017-02-03T10:37:31.691Z..2017-02-03T10:37:50.987Z)	True	False	EARLY	13	13	13
3	[2017-02-03T10:37:31.691Z..2017-02-03T10:38:03.287Z)	True	False	EARLY	28	28	28
4	[2017-02-03T10:37:31.691Z..2017-02-03T10:38:20.673Z)	True	False	EARLY	43	43	43
5	[2017-02-03T10:37:31.448Z..2017-02-03T10:38:26.637Z)	True	False	LATE	58	58	58
6	[2017-02-03T10:37:31.448Z..2017-02-03T10:38:26.637Z)	False	True	LATE	60	60	60

We can use this table to learn about windows and triggers.

How does AfterEach.inOrder works? Does it switch triggers after every message?

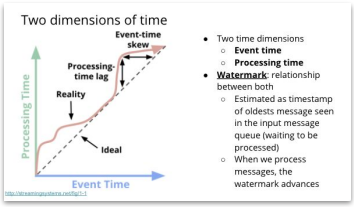
Once a trigger is fired, the next one is used.

If we don't use the Repeteadly in the last trigger, then we would not be triggering any more and no output would be produced. That would be similar to dropping data.

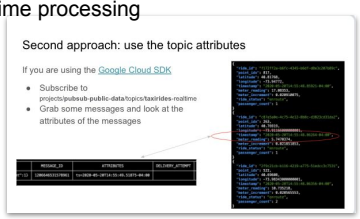
Because we are using count based triggers, there is no firing at the watermark (ON_TIME).

Take away

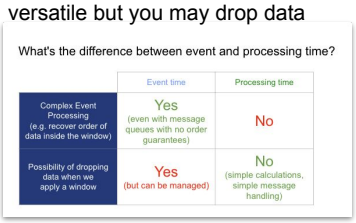
Event time and processing time



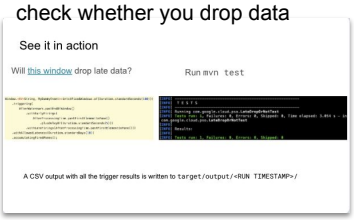
Use the right timestamps for event time processing



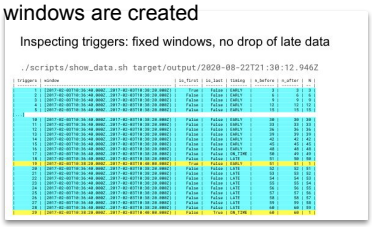
Event time and windows, more versatile but you may drop data



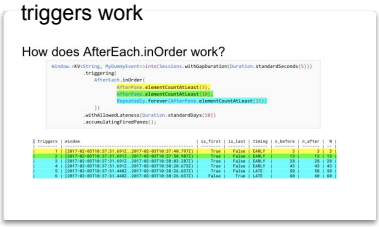
TestStream and unit testing help to check whether you drop data



Inspect the output to see how the windows are created



Understand how different types of triggers work



Israel Herraiz ihr@google.com @herraiz https://github.com/iht/beam-late-data

Event time and processing time. There are actually two dimensions of time, and if we don't explicitly choose to work on event time, we will likely be working on processing time.
The watermark is the most important concept to understand late data. Data is not late per-se. Data is late when it is compared against the watermark. The watermark is estimated using the backlog of messages waiting to be processed, and increases monotonically.

To work in event time, we need to use a timestamp attribute. The details are different depending on the message queue. But we need a timestamp defined in the origin of the message.

What's the matter with event time? We can only do complex event processing (e.g. recovering the order of data in a window) if we work on event time. Working in processing time is similar to micro-batching. Safe but simple and not so interesting. Working on event time we can write highly sophisticated pipelines with high performance and low latency. But we risk dropping late data.

We can use unit testing and TestStream to evaluate whether a particular window, with specific properties, would drop our late data, and what are the limits of late data that can be tolerated.

We can also use that unit test to inspect how the elements are aggregated to a window, and how the triggers are producing the output.

If we are not sure about a particular window option or trigger, we can always just add the window to this unit test, and have a look at the output to get a clear idea of how does it work.